

PRAKTIKUM SISTEM BASIS DATA

Nama	Nelson Laurensius	No. Modul	6
NPM	2306161845	Tipe	Tugas Tambahan

1. Lanjutkan backend sesuai dokumentasi
2. response yang telah ditambah

Method	Endpoint	Success	Failed
GET	/transaction	<pre> { "success": true, "message": "Transaction found", "payload": [{ "id": "5d75076-c92f-4d42-908d-e908195a4b1", "user_id": "a80c2e2ce-6d3c-455c-821e-a180920e9f9", "item_id": "a055669f-87ca-4d53-9035-861db0f92ba2", "quantity": 5, "total": 500000, "status": "paid", "created_at": "2025-03-26T01:40:42.293Z" }] } </pre>	<pre> { "success": false, "message": "Transaction not found", "payload": null } </pre>
DELETE	/transaction/:id	<pre> { "success": true, "message": "Transaction deleted", "payload": { "id": "5d75076-c92f-4d42-908d-e908195a4b1", "user_id": "a80c2e2ce-6d3c-455c-821e-a180920e9f9", "item_id": "a055669f-87ca-4d53-9035-861db0f92ba2", "quantity": 5, "total": 500000, "status": "paid", "created_at": "2025-03-26T01:40:42.293Z" } } </pre>	<pre> { "success": false, "message": "Transaction not found", "payload": null } </pre>

3. tingkatkan kode scalability, security, dan error handling
4. apa saja yang sudah saya lakukan untuk scalability, security, dan error handling

Tipe	Implementasi	Deskripsi
Error Handling	<pre> { "success": false, "message": "All fields are required", "payload": null } </pre>	Error handling supaya setiap fields di create dan update item harus terisi

Error Handling	<pre>{ "success": false, "message": "There are no items found", "payload": null }</pre>	Error handling supaya jika tidak ada item saat get All akan menampilkan tulisan
Error Handling	<pre>{ "success": false, "message": "No items found in this store", "payload": null }</pre>	Error handling supaya ketika get item in store tapi storenya kosong itemnya akan menampilkan tulisan
Error Handling	<pre>{ "success": false, "message": "Item not found", "payload": null }</pre>	Error handling item not found di update item
Error Handling	<pre>{ "success": false, "message": "There are no stores found", "payload": null }</pre>	error handling there are stores not found jika tidak ada store ketika get all stores
Error Handling	<pre>{ "success": false, "message": "Store name and address are required" }</pre>	error handling pada update store agar sama seperti saat create dimana semua fields harus terisi
Error Handling	<pre>{ "success": false, "message": "Name, email, and password are required", "payload": null }</pre>	error handling saat register dan login dan juga update agar semua fields harus terisi

Error Handling	<pre>{ "success": false, "message": "Invalid email format", "payload": null }</pre>	error handling agar format email harus benar sesuai regex
Error Handling	<pre>{ "success": false, "message": "User ID and amount are required", "payload": null } { "success": false, "message": "Top up failed", "payload": null }</pre>	Error handling saat topup bahwa user id dan amount harus terisi dan jika user nya salah, maka akan failed
Error Handling	<pre>{ "success": false, "message": "Insufficient balance", "payload": null }</pre>	error handling jika balance tidak cukup ketika melakukan payment
Error Handling	<pre>{ "success": false, "message": "Transaction already paid", "payload": null }</pre>	error handling jika sudah dibayar dan ingin bayar lagi
Error Handling	<pre>{ "success": false, "message": "Transaction not found", "payload": null }</pre>	error handling jika transaction tidak ditemukan
Error Handling	<pre>{ "success": false, "message": "Price cannot be negative", "payload": null }</pre>	error handling harga item tidak boleh negative
Scalability	<pre>const pool = new Pool({</pre>	Menambahkan

	<pre> connectionString: process.env.PG_CONNECTION_STRING, ssl: { rejectUnauthorized: false, }, max: 20, idleTimeoutMillis: 30000, }); const query = async (text, params) => { let client; try { client = await pool.connect(); const res = await client.query(text, params); return res; } catch (error) { console.error("Error executing query", error); throw error; } finally { if (client) { client.release(); } } }; </pre>	Connection pooling diterapkan dengan max dan idle timeout millis untuk menentukan jumlah maks koneksi yang dapat dibuat dalam pool dan menentukan waktu sebelum idle ditutup. Selain itu juga membuat variable client untuk menyimpan instance koneksi dari pool. dan menambahkan finally untuk memastikan client.release() dipanggil untuk return koneksi ke pool. Semua hal tersebut dilakukan supaya koneksi database hanya dibuka ketika digunakan, ini dapat mengurangi beban database dan juga efesiesnsi sumber daya pada database
Security	<pre> PS D:\Documents\KULIAH\Semester4\SBDPrak\5\CS_KL_NelsonLaurensius_2306161845_SBD6> npm install jsonwebtoken authMiddleware.js: const jwt = require("jsonwebtoken"); const baseResponse = require("../utils/baseResponse.util"); </pre>	meningkatkan security dengan membuat otentikasi dan otorisasi pada user supaya setiap login perlu token

	<pre>const authMiddleware = (req, res, next) => { const token = req.header("Authorization")?.replace("Bearer ", ""); console.log("Token:", token); // Tambahkan log ini if (!token) { return baseResponse(res, false, 401, "Authentication required", null); } try { const decoded = jwt.verify(token, process.env.JWT_SECRET); req.user = decoded; next(); } catch (error) { console.error("Error verifying token:", error); // Tambahkan log ini return baseResponse(res, false, 401, "Invalid token", null); } }; module.exports = authMiddleware; authorizeMiddleware.js: const baseResponse = require("../utils/baseResponse.util"); const authorizeMiddleware = (roles) => (req, res, next) => { if (!roles.includes(req.user.role)) { return baseResponse(res, false, 403, "Forbidden", null); } }</pre>	yang valid.
--	---	-------------

```

    next();
  };

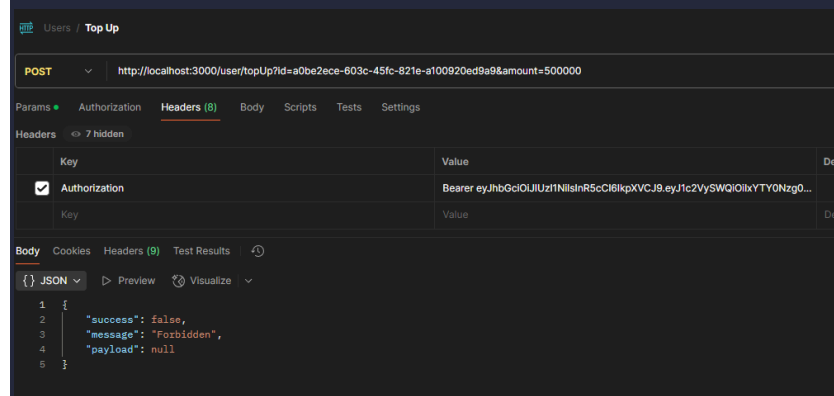
module.exports = authorizeMiddleware;

.env:
JWT_SECRET = "W0hnZ5UukxWZlSpDVdqIcV4VlDq2aLsQ
PXGgeXW28kzcu67XCkkMgyLlpgqFITiv
noOT6RD1cVzPPPKAJu2ZD8AQWlsYvxbU
gU82jb3la5HsZpZAMknnLgvb7gpJapHC
ulW6ao4JHczxR0tmiS2Lxlo95zSZ6Nlo
16IdZiPqISHQLn5a2b5fyWNBbZPhnPY0
OdvtYCG4iIoTKsdEGShnB3LVWbpL9qvX"

user.controller.js:
const token = jwt.sign(
  { userId: user.id, email: user.email, role:
user.role || "user" },
  process.env.JWT_SECRET,
  { expiresIn: "1h" }
);

jika token login sebagai user:

```



Users / Top Up

POST http://localhost:3000/user/topUp?id=a0be2ece-603c-45fc-821e-a100920ed9a9&amount=500000

Params Authorization Headers (8) Body Scripts Tests Settings

Headers <> 7 hidden

Key	Value	Description
☒ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiIxYTU0Nzg0...	
Key	Value	Description

Body Cookies Headers (9) Test Results <>

{ JSON > Preview <> Visualize <>

```

1 {
2   "success": false,
3   "message": "Forbidden",
4   "payload": null
5 }

```

jika token login sebagai admin:

	<pre> { "success": true, "message": "Top up successful", "payload": { "id": "a0be2ece-603c-45fc-821e-a100920ed9a9", "name": "nelson Updated", "email": "lauren@mail.com", "password": "\$2b\$10\$/3nGPJYxaVpwWmJ3tcEHpuFXpSQLKANJMt.5Ux5JozZ8Cwt17dqb6", "balance": 1500000, "created_at": "2025-03-18T21:58:26.746Z", "role": "user" } } </pre> <pre> ALTER TABLE users ADD COLUMN role VARCHAR(255) DEFAULT 'user'; </pre>	
security	<pre> const fileFilter = (req, file, cb) => { if (file.mimetype.startsWith("image/")) { cb(null, true); } else { cb(new Error("Only image files are allowed!"), false); } }; const upload = multer({ storage: storage, fileFilter: fileFilter, limits: { fileSize: 5 * 1024 * 1024 }, }); </pre>	Membatasi ukuran file yang diupload agar mencegah DoS